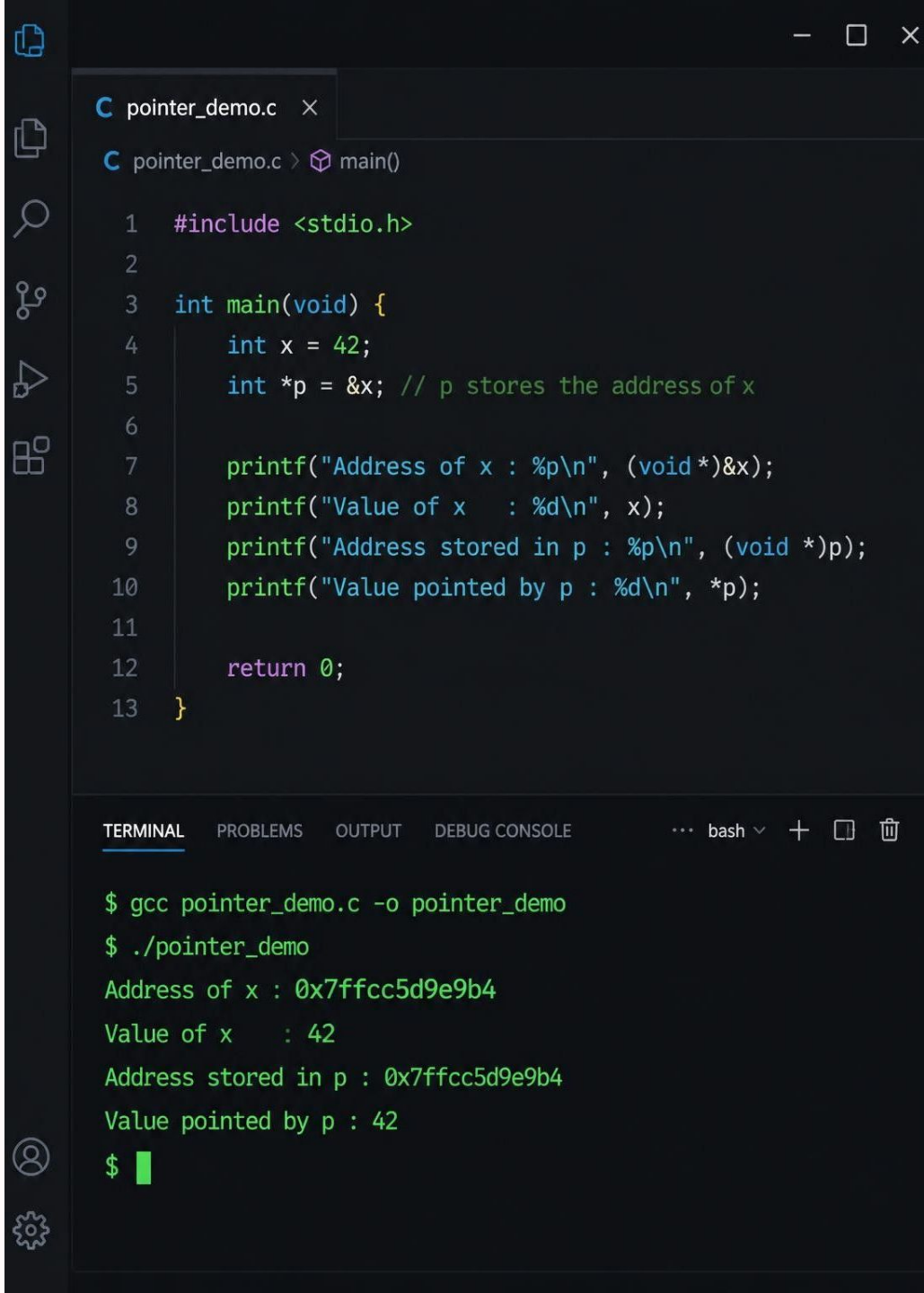
Introduzione ai Puntatori in C

Indirizzi e memoria, dichiarazione e dereferenziazione, errori e puntatori sicuri

LINGUAGGIO C | PUNTATORI | MEMORIA

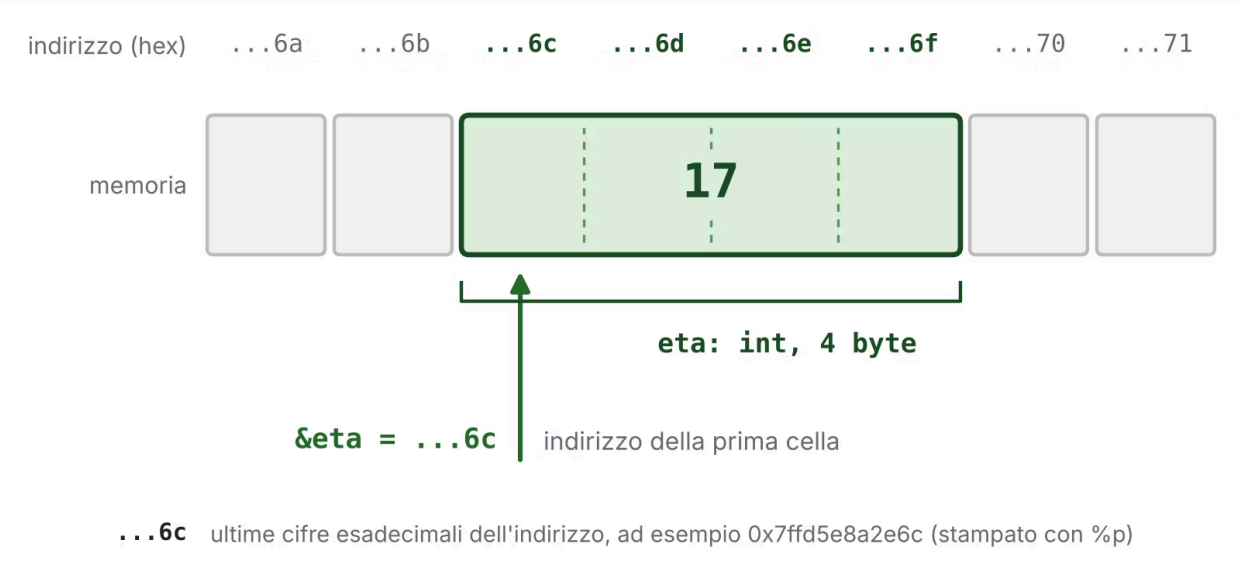


La Memoria e gli Indirizzi

La memoria del computer è una lunga sequenza di celle da 1 byte. Ogni cella ha un numero: il suo indirizzo. Una variabile occupa alcune celle consecutive e il suo indirizzo è quello della prima.

Elemento	Significato
&x	indirizzo della variabile x
%p	specifica di printf per gli indirizzi
(void *)	conversione richiesta da %p

Gli indirizzi cambiano a ogni esecuzione: il sistema operativo colloca il programma in punti diversi della memoria.




```
int eta = 17;
printf("%p\n", (void *) &eta);
```

scanf e l'Operatore &

Il problema: scanf deve scrivere il valore letto in una variabile del chiamante. Per farlo le serve sapere dove si trova quella variabile.

La soluzione: si passa a scanf l'indirizzo della variabile.

Scrittura	Effetto
`scanf("%d", &eta)`	scrive in eta
`scanf("%d", eta)`	passa il VALORE di eta come se fosse un indirizzo: scrittura in un punto casuale

Con -Wall il compilatore segnala la & mancante: gli avvisi vanno sempre letti e corretti.

```
scanf("%d", &eta);          /* indirizzo di eta */
scanf("%19s", nickname);    /* nickname è un array:
                               il nome indica già
                               l'indirizzo del primo elemento */
```

sizeof: Quanto Occupa una Variabile

```
sizeof(int)          /* dimensione del tipo */
sizeof(media)        /* dimensione della variabile */
sizeof(v)            /* dimensione dell'intero array */
n = sizeof(v) / sizeof(v[0]);   /* numero di elementi */
```

Esempio: un datalogger con 32 KB registra per 9.5 giorni con misure in double, per 45.5 giorni con interi in decimi di grado.

Le dimensioni non sono fissate dal linguaggio: si usa sempre sizeof invece di scrivere 4 o 8 a mano.

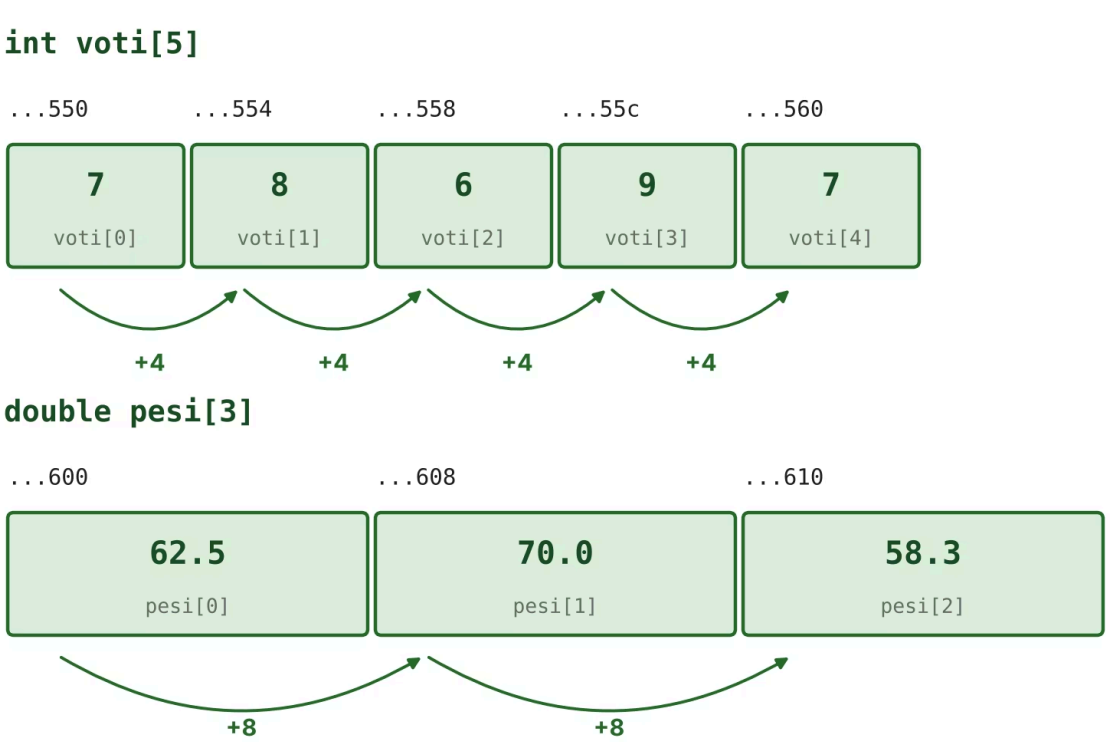
Tipo	Byte tipici (64 bit)
`char`	1 (garantito)
`short`	2
`int`, `float`	4
`long`, `double`	8
indirizzo (qualsiasi tipo)	8

Variabili e Array in Memoria

- L'ordine delle variabili in memoria è scelto dal compilatore: non si può fare alcuna ipotesi
- Gli elementi di un array sono sempre consecutivi

Confronto	Domanda
a == b	le due variabili hanno lo stesso valore?
&a == &b	sono la stessa variabile?

Esempio: un trasferimento di credito da una tessera a se stessa si riconosce confrontando gli indirizzi, non i saldi.



Indirizzi esadecimali abbreviati alle ultime cifre: ...558 + 4 = ...55c (dopo 9 vengono a, b, c, d, e, f).

Il Puntatore

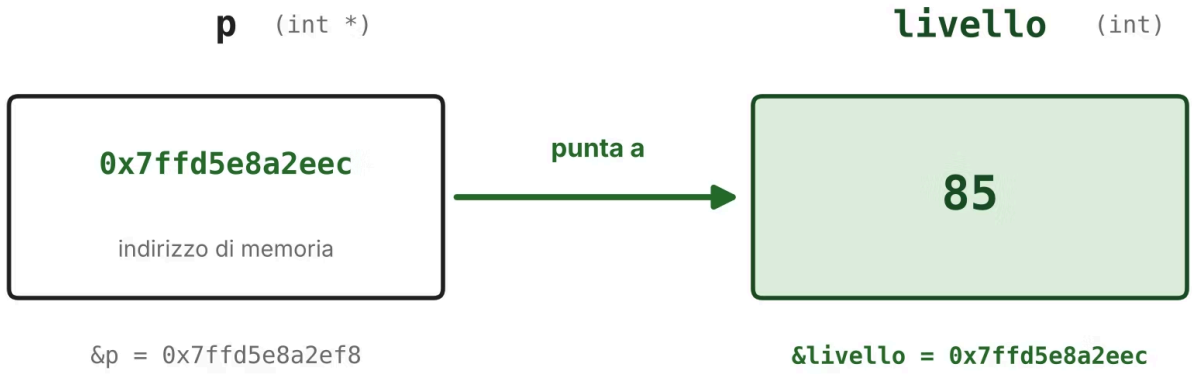
Un puntatore è una variabile che contiene un indirizzo.

Espressione	Significato
`int *p`	dichiarazione di un puntatore a int
`p = &x`	p contiene l'indirizzo di x
`*p`	la variabile puntata: si legge e si scrive

Il simbolo * ha due significati: nella dichiarazione dice che p è un puntatore, in un'espressione indica la variabile puntata.

```
int livello = 70;
int *p;          /* p: puntatore a int */

p = &livello;     /* p punta a livello */
*p = 85;          /* livello vale 85 */
x = *p;           /* legge livello */
```



`0x7ffd5e8a2eec`	indirizzo scritto in esadecimale (prefisso 0x, cifre 0-9 e a-f), come lo stampa %p
`*p`	la variabile all'indirizzo contenuto in p, cioè livello

Copiare Puntatori e Copiare Valori

```
cassa_1 = &sala_a;
cassa_2 = cassa_1;     /* stesso indirizzo */
*cassa_1 = *cassa_1 - 4;
*cassa_2 = *cassa_2 - 2;   /* sala_a: 120 - 4 - 2 */

cassa_2 = &sala_b;     /* punta a un'altra variabile */
*cassa_2 = *cassa_1;   /* copia il valore */
```

Istruzione	Effetto
`q = p`	q punta alla stessa variabile di p
`*q = *p`	la variabile puntata da q prende il valore di quella puntata da p
`p = &y`	p punta a un'altra variabile

Se due puntatori indicano la stessa variabile, una modifica fatta tramite uno si vede anche tramite l'altro.

A Che Cosa Serve un Puntatore

Il problema: il pannello di un termostato ha un solo paio di tasti + e − per tre zone. Senza puntatori servirebbero tre copie delle stesse istruzioni, una per zona.

La soluzione: un puntatore indica la variabile "selezionata" e le istruzioni si scrivono una volta sola.

- Il giocatore di turno in una gara di dadi
- Il salvadanaio scelto: attivo = &saldi[k]
- La scorta più bassa tra tre variabili
- Il miglior tempo di una gara

Un puntatore a un elemento di un array (&saldi[k]) si confronta con &saldi[j] per sapere quale elemento è selezionato.

```
int *zona_attiva = NULL;

if (comando == 'S') {
    zona_attiva = &soggiorno;
} else if (comando == 'C') {
    zona_attiva = &camera;
}
...
*zona_attiva = *zona_attiva + PASSO;
```

NULL: Nessuna Variabile

Il problema: un puntatore dichiarato e non inizializzato contiene un indirizzo casuale. Usarlo significa leggere o scrivere in un punto qualsiasi della memoria.

La soluzione: quando un puntatore non punta ancora a nulla gli si assegna NULL, e prima di usarlo si controlla.

- NULL per "nessuna selezione": codice inesistente, badge non letto
- NULL per "nessun risultato": nessun atleta ha concluso la gara

Regola: ogni puntatore si inizializza alla dichiarazione, con un indirizzo valido oppure con NULL.

```
int *scelta = NULL;

if (codice == 'C') {
    scelta = &caffe;
}

if (scelta != NULL) {
    *scelta = *scelta + 1;
} else {
    printf("codice inesistente\n");
}
```

I Quattro Errori Tipici

Il problema: spesso il programma non si ferma subito. Scrive in un punto sbagliato e l'errore si manifesta più tardi, in un'altra parte del codice.

La soluzione: due regole.

1. Inizializzare sempre i puntatori.
2. Una funzione che riceve un puntatore che può essere NULL lo controlla prima di usarlo.

Il blocco { } di una variabile ne delimita la vita: fuori dal blocco il suo indirizzo non va più usato.

Errore	Codice sbagliato	Conseguenza
Puntatore non inizializzato	`int *p; *p = 5;`	scrittura in un punto casuale
Dereferenziazione di NULL	`p = NULL; *p = 5;`	interruzione del programma
scanf senza &	`scanf("%d", n);`	scrittura in un punto casuale
Variabile che non esiste più	`{ int t; p = &t; } *p = 1;`	indirizzo non più valido

NULL come Scelta: Parametri Facoltativi

Il problema: una funzione calcola minimo, massimo e media dei voti, ma non tutti i chiamanti hanno bisogno di tutti i risultati.

La soluzione: chi non vuole un risultato passa NULL; la funzione scrive solo nei puntatori diversi da NULL.

Se n vale 0 la funzione non scrive nulla e restituisce 0: la media di zero voti non esiste.

```
int statistiche(const int voti[], int n,
                int *minimo, int *massimo, double *media);

statistiche(v, n, &min, &max, &media);  /* tutto */
statistiche(v, n, NULL, NULL, &media);  /* solo la media */
statistiche(v, n, &min, NULL, NULL);    /* solo il minimo */

if (media != NULL) {
    *media = (double) somma / n;
}
```

Il Tipo del Puntatore

Tutti gli indirizzi hanno la stessa dimensione, ma *p legge sizeof(*p) byte e li interpreta secondo il tipo.

Puntatore	*p legge
`int *pi`	4 byte come intero
`double *pd`	8 byte come numero con virgola
`char *pc`	1 byte come carattere

Copiare valori tra tipi diversi è una normale conversione; far puntare un int * a un double è un errore.

```
double media = 7.5;
int *p = &media;      /* avviso: tipi incompatibili */

*pd = *pi;            /* corretto: copia di un valore,
                         l'int diventa double */
*pi = (int) *pd;      /* la parte decimale si perde */
```

const e Puntatori

```c
int limita(int *valore, const int *massimo)
{
    if (*valore > *massimo) {
        *valore = *massimo;     /* permesso */
    }
    /* *massimo = 0;   errore di compilazione */
}

int *const sensore = &pressione;        /* sempre la stessa variabile */
const int *const soglia = &ARRESTO;     /* nulla si può cambiare */
```

Dichiarazione	Si può cambiare *p?	Si può cambiare p?
`const int *p`	no	sì
`int *const p = &x`	sì	no
`const int *const p = &x`	no	no

Nei parametri, const int * dice che la funzione legge la variabile del chiamante ma non la modifica: un errore di logica diventa un errore di compilazione.

Errori Comuni da Evitare

1

Dimenticare & in scanf.

2

Stampare un indirizzo con %d invece che con %p e (void *).

3

Supporre l'ordine delle variabili in memoria o la dimensione dei tipi.

4

Dereferenziare un puntatore non inizializzato.

5

Dereferenziare NULL senza controllo.

6

Confondere p (indirizzo) e *p (variabile puntata).

7

Far puntare un int * a una variabile di un altro tipo.

8

Conservare l'indirizzo di una variabile che non esiste più.

Riepilogo Concetti

Operazione	Tecnica	Note
Indirizzo di x	`&x`	stampato con %p e (void *)
Dimensione	`sizeof`	tipi, variabili, array
Dichiarare un puntatore	`int *p`	una variabile per riga
Puntare a x	`p = &x`	stesso tipo di x
Leggere/scrivere	`*p`	dereferenziazione
Nessuna variabile	`NULL`	controllare prima dell'uso
Stessa variabile?	`p == q, &a == &b`	confronto di indirizzi
Risultati facoltativi	parametro NULL	la funzione controlla
Sola lettura	`const int *p`	parametri protetti
Puntatore fisso	`int *const p`	punta sempre alla stessa variabile

Esempi Pratici: Indirizzi e Memoria

Concetti base — tre file su indirizzi e dimensioni.

1 – 010-indirizzo-di-una-variabile.c

Problema: dove si trova una variabile.

Tecnica: &, %p, indirizzo costante al cambiare del valore, scanf.

2 – 020-sizeof-e-tipi.c

Problema: quanto occupa ogni tipo.

Tecnica: sizeof di tipi, array, stringhe e indirizzi.

3 – 030-variabili-e-array-in-memoria.c

Problema: come è disposto un array.

Tecnica: elementi consecutivi in array di int e di double.

Esempi Pratici: Problemi su Indirizzi e Memoria

040 – Mappa della memoria

Tipo, indirizzo e byte dei dati di una scheda studente e memoria per 1000 schede.

050 – Memoria del datalogger

Scelta dei tipi per registrare più giorni di misure meteo in 32 KB.

060 – Lettura con scanf

Modulo di iscrizione a un torneo: perché scanf vuole gli indirizzi.

070 – Salvataggio della partita

Dimensione del salvataggio e slot disponibili, livelli in int o in char.

080 – Trasferimento tra tessere

Stesso valore o stessa variabile: il confronto degli indirizzi.

SEZIONE 2

Esempi Pratici: Dichiarazione e Dereferenziazione

Concetti base — tre file sull'uso dei puntatori.

1 – 010-dichiarazione-e-uso.c

Problema: leggere e scrivere tramite un indirizzo.

Tecnica: int *p, p = &x, *p in lettura e scrittura.

2 – 020-piu-puntatori-stessa-variabile.c

Problema: due casse che vendono per la stessa sala.

Tecnica: copia di puntatori, cambio di variabile, copia del valore.

3 – 030-puntatore-null.c

Problema: codice di una bevanda inesistente.

Tecnica: NULL come "nessuna selezione" e controllo.

SEZIONE 2

Esempi Pratici: Problemi con i Puntatori

040 – Termostato

Tasti + e − che agiscono sulla zona selezionata.

050 – Giocatore di turno

Punteggio e nome del giocatore di turno tramite puntatori.

060 – Salvadanaio attivo

Versamenti e prelievi sull'elemento selezionato di un array.

070 – Riordino delle scorte

Puntatore alla scorta più bassa di un distributore automatico.

080 – Record della gara

Puntatore al miglior tempo, NULL se nessuno, record aggiornato.

SEZIONE 3

Esempi Pratici: Errori e Puntatori Sicuri

Concetti base — tre file sull'uso corretto dei puntatori.

1 – 010-non-inizializzato-e-null.c

Problema: tornelli con badge non letti.

Tecnica: inizializzazione e controllo di NULL nelle funzioni.

2 – 020-tipi-dei-puntatori.c

Problema: che cosa legge *p.

Tecnica: sizeof(*p), tipi incompatibili, copia di valori tra tipi.

3 – 030-puntatori-const.c

Problema: proteggere dati da modifiche.

Tecnica: const int *, int *const, parametri const.

SEZIONE 3

Esempi Pratici: Problemi sui Puntatori Sicuri

040 – Configurazione in sola lettura

Limiti di una cassa audio passati come const int *.

050 – Statistiche facoltative

Parametri OUT facoltativi: NULL per i risultati non richiesti.

060 – Correzione degli errori

I quattro errori tipici riconosciuti e corretti nel programma di una palestra.

070 – Selezione sicura

Impostazioni di una lavatrice con puntatori e limiti controllati.

080 – Limiti di sicurezza

Pressa industriale: puntatore costante al sensore e soglie di sola lettura.